

# 数据结构与算法期末试卷

(回忆版)

考试时间：120 分钟

**前言：**本试卷为回忆版，由于记忆较为模糊，文字表述可能与原始试卷存在差异。不保证选项顺序和具体表述完全一致，但题目考查的知识点和内容范围是完全一致的。请读者理解并酌情参考使用。

## 一、选择题

1. 下列关于算法特性的描述，正确的是 ()
  - A. 算法的可行性是指算法必须在执行有限步骤后结束，不能无限循环。
  - B. 算法的确定性是指算法的每一步骤必须有确切的定义，不会出现二义性，在任何条件下，算法的执行路径都应该是明确的。
  - C. 正确性，高效性属于算法的五大基本特性。
  - D. 输入这一算法基本特性要求算法至少有一个输入。
2. 广度优先搜索 (BFS) 通常使用以下哪种数据结构？
  - A. 栈
  - B. 队列
  - C. 优先队列
  - D. 堆
3. C 语言中，二维数组  $A[10][10]$ ，每个元素占 1 个字节，基地址为 1000，采用行优先存储，则元素  $A[3][5]$  的地址是 ()
  - A. 1015
  - B. 1024
  - C. 1035
  - D. 1053
4. 给定有向图  $G$ ，顶点集  $\{1,2,3,4,5,6\}$ ，边集及权值： $1 \rightarrow 2(4)$ ， $1 \rightarrow 3(2)$ ， $3 \rightarrow 2(1)$ ， $2 \rightarrow 4(3)$ ， $3 \rightarrow 4(5)$ ， $3 \rightarrow 5(2)$ ， $4 \rightarrow 6(1)$ ， $5 \rightarrow 6(2)$ 。从顶点 1 到顶点 6 的最短路径长度是 ()
  - A. 5
  - B. 6

- C. 7  
D. 8
5. 给定权值集合  $\{1, 2, 3, 5, 6, 7, 10\}$ ，构造 Huffman 树，其最小带权路径长度为 ()  
A. 34  
B. 74  
C. 88  
D. 95
6. 对数组  $[4, 6, 3, 5, 1, 7]$  进行建大顶堆 (heapify) 操作，得到的堆是 ()  
A.  $[7, 5, 6, 4, 1, 3]$   
B.  $[7, 6, 4, 5, 1, 3]$   
C.  $[7, 6, 4, 5, 3, 1]$   
D.  $[7, 6, 3, 5, 4, 3]$
7. 设有活动集合： $\{(5, 7), (2, 13), (1, 4), (0, 6), (12, 14), (3, 9), (6, 10), (8, 12), (5, 9), (3, 5), (8, 11)\}$ ，其中每个活动用 (开始时间, 结束时间) 表示。最多能安排的活动数量是 ()  
A. 3  
B. 4  
C. 5  
D. 6
8. 将元素序列 1, 3, 7, 6, 4, 5, 2, 8 依次插入初始为空的 AVL 树，插入完成后树的根节点是 ()  
A. 3  
B. 4  
C. 5  
D. 6
9. 下列关于关键路径的描述，正确的是 ()  
A. 关键活动的最早开始时间小于最晚开始时间  
B. 关键活动的最早开始时间与最晚开始时间相等  
C. 关键路径是网络图中从源点到汇点的最短路径  
D. 一个工程只有一条关键路径
10. 下列递推式，不能使用课内提到的主定理的三种情况解决的是  
A.  $T(n) = 2T(n/2) + n \log n$   
B.  $T(n) = 3T(n/4) + n$   
C.  $T(n) = 4T(n/2) + n^2$   
D.  $T(n) = T(n/2) + n \log n$

## 二、填空题

1. 设有一个  $n$  阶对称矩阵  $A$ ，采用压缩存储方式存储下三角部分（包括对角线），按行优先存储。若矩阵索引从 1 开始，数组索引从 1 开始，则元素  $A[3][3]$  在一维数组中的下标为\_\_\_\_\_。
2. 已知广义表  $A=((a,b),d,e)$ ，则  $\text{Head}(\text{Tail}(\text{Head}(A)))$  的结果是\_\_\_\_\_。
3. 在一棵二叉树中，度为 2 的结点个数为  $k$ ，则叶子结点的个数为\_\_\_\_\_。
4. 已知二叉树的中序遍历序列为：DBFGEACHI，后序遍历序列为：DGFEBIHCA，则前序遍历序列为\_\_\_\_\_。
5. 一个森林中有 100 个结点，80 条边，则该森林中共有\_\_\_\_\_棵树。
6. 对一棵树采用孩子兄弟表示法存储后，如果对二叉树进行中序遍历，则相当于对原树进行\_\_\_\_\_遍历。
7. 在对二叉树进行后序线索化时，若某结点的右孩子为空，则其右线索指向该结点的\_\_\_\_\_。
8. 设有 01 背包问题：背包容量为 10，物品重量分别为  $\{1,2,3,5,7\}$ ，价值分别为  $\{1,1,5,2,4\}$ 。则能获得的最大价值为\_\_\_\_\_。
9. 假设有有序数组为  $\{1,2,3,4,5,6,7,8,9,10\}$ ，采用二分查找，则查找成功时的平均查找长度（ASL）为\_\_\_\_\_。
10. 给定数组  $\{5,3,8,6,2,7,1,4\}$ ，以第一个元素 5 为枢轴，进行一趟快速排序（partition）操作后，数组变为\_\_\_\_\_。

## 三、简答题

1. 设有向图  $G$  的顶点集为  $\{1,2,3,4,5,6\}$ ，边及权值如下： $1 \rightarrow 2(5)$ ， $1 \rightarrow 3(2)$ ， $2 \rightarrow 3(-10)$ ， $3 \rightarrow 4(1)$ ， $4 \rightarrow 5(1)$ ， $5 \rightarrow 6(1)$ 。
  - (1) 使用 Dijkstra 进行模拟，求得到的最短路径长度；
  - (2) 真实的最短路径长度为？解释为什么 Dijkstra 失效。
2. 设哈希表长度  $\text{HASHTABLE\_SIZE} = 11$ ，关键字序列为  $\{23, 34, 56, 78, 11, 45\}$ 。
  - (1) 若开放定址法，选择平方增量  $(1, -1, 4, -4, \dots)$ ，计算最终结果，并计算 ASL；
  - (2) 若使用链地址法，计算最终结果，并计算 ASL。

## 四、手写代码题

### 1. 基于栈的拓扑排序。

(1) 假设有向图有 5 个顶点 (0,1,2,3,4) 和边: 0->1, 0->2, 1->3, 2->3, 3->4。根据基于栈的拓扑排序算法, 计算拓扑序列。

(2) 补全代码实现。

```
1 // 边节点
2 typedef struct ArcNode {
3     int adjvex;           // 该边指向的顶点在数组中的下标
4     struct ArcNode *nextarc; // 指向下一条边的指针
5 } ArcNode;
6 // 顶点节点
7 typedef struct VNode {
8     int data;             // 顶点信息
9     ArcNode *firstarc;    // 指向第一条依附该顶点的边的指针
10 } VNode;
11 // 图结构
12 typedef struct ALGraph{
13     VNode vertices[100]; // 邻接表数组
14     int vexnum, arcnum;  // 顶点数和边数
15 } ALGraph;
16
17 int* topologicalSort(ALGraph* G) {
18     int n = G->vexnum;
19     int* inDegree = (int*)calloc(n, sizeof(int));
20     int* result = (int*)malloc(n * sizeof(int));
21     int stack[1000];
22     int top = -1; // 栈顶指针
23     for (int i = 0; i < n; i++) {
24         ArcNode* p = G->vertices[i].firstarc;
25         while (p != NULL) {
26             inDegree[p->adjvex]++;
27             p = p->nextarc;
28         }
29     }
30     for (int i = 0; i < n; i++) {
31         if (inDegree[i] == 0) {
32             stack[++top] = i;
33         }
34     }
```

```

34     }
35     int count = 0;
36     while (top != -1) {
37         int u = stack[top--];
38         result[count++] = u;
39         ArcNode* p = G->vertices[u].firstarc;
40         while (p != NULL) {
41             int v = p->adjvex;
42             inDegree[v]--;
43             if (_____) { // 补充该部分代码
44                 stack[++top] = v;
45             }
46             p = p->nextarc;
47         }
48     }
49     free(inDegree);
50     if (count < n) {
51         // 图中有环，无法生成完整的拓扑序列
52         free(result);
53         return NULL;
54     }
55     return result;
56 }

```

## 2. (背景介绍，主要是文件组织形式为树结构)

(1) 某个树，节点 1 有孩子 2,3,4；节点 2 有孩子 5,6；节点 3 有孩子 7,8；节点 5 有孩子 9。通过孩子兄弟表示法，转换为二叉树。

(2) 编写代码，遍历一个孩子兄弟表示法表示的树，计算叶子结点个数。

```

1 typedef struct TreeNode {
2     int val;
3     TreeNode* firstChild; // 指向第一个孩子
4     TreeNode* nextSibling; // 指向下一个兄弟
5 } TreeNode;
6
7 int countLeaves(TreeNode* root) {
8     // 待编写
9 }

```

3. 图有  $n$  个顶点，邻接矩阵为  $\text{graph}[n][n]$ ，如果边不存在则用一个极大值  $\text{INF}$  表示。

基于此，实现 Prim 算法。

```
1 int prim(int n, int** graph) {  
2     int lowcost[n]; // 最小代价数组  
3     // 待编写  
4 }
```

4. 要计算某字符串 s 的最长回文子串的长度。

- (1) 构建 dp 数组，写出转移方程和边界条件；
- (2) 编写代码。

```
1 int longestPalindromeSubstring(int n, char* s) {  
2     // 待编写  
3 }
```

5. 使用回溯法，从 n 个数的数组中选 k 个数，判断是否存在一种选择使得总和为 T。

- (1) 列出四种剪枝策略。该算法的时间复杂度是 \_\_\_\_\_；
- (2) 编写代码。

```
1 // 待编写
```